



FIREBRAND

LYCEUM EDUCATION GROUP

Introduction to Power Platform and Event-Driven Logic

Power Platform Pathway | Session 1

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, **S7**, S10,
S11, S12, S16

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Introduction to Power Platform and Event-Driven Logic

Lesson Objectives

Apply

- Apply event-driven programming concepts within Power Apps

Demonstrate

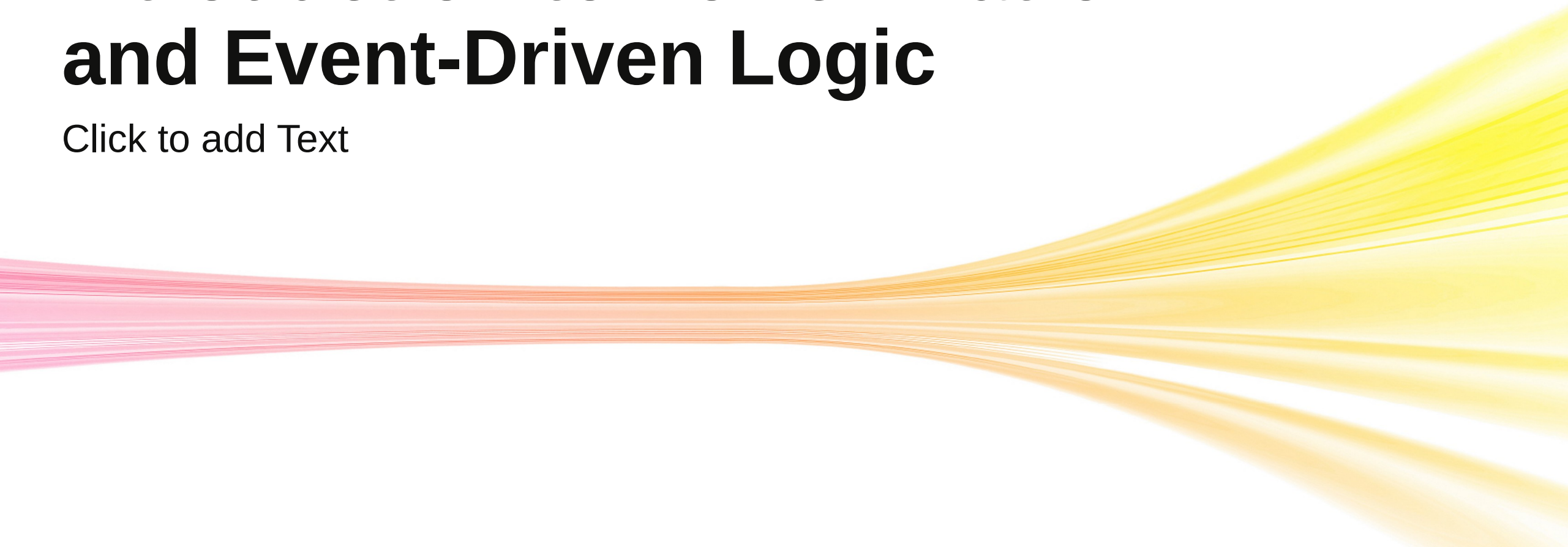
- Demonstrate how user actions trigger logic

Analyse

- Analyse app structure and identify input/output flow

Introduction to Power Platform and Event-Driven Logic

Click to add Text

A decorative graphic consisting of multiple thin, overlapping wavy lines that create a sense of motion. The lines are primarily pink and yellow, with some orange tones, and they originate from the left side of the slide, fanning out towards the right.

The Power Platform Ecosystem

**Power
Apps**

**Power
Automate**

Dataverse

Power Apps – The Front End

- **Low-Code UI:** Build custom apps visually
- **Drag-and-Drop:** Design interfaces without writing complex code
- **Cross-Platform:** Run apps on mobile, tablet, or web
- **Data Connections:** Connect seamlessly to hundreds of data sources

Power Automate – The Logic

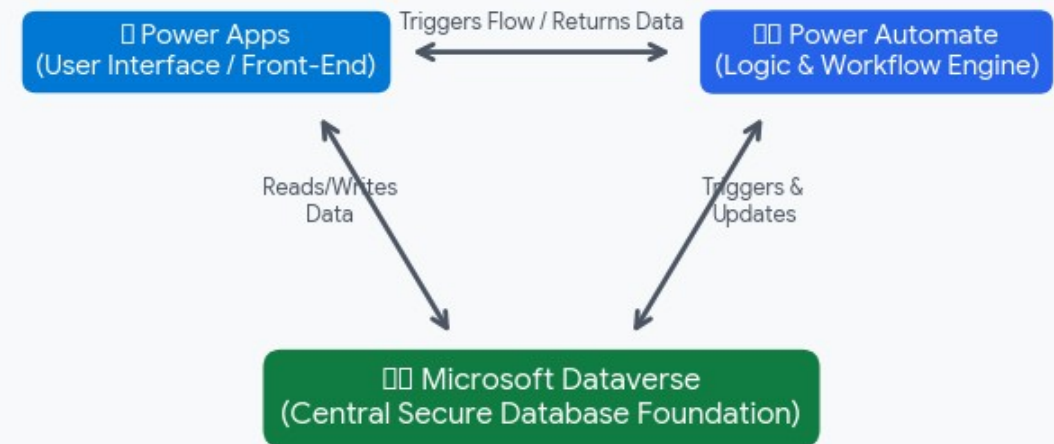
- **Workflow Automation:** Eliminate repetitive digital tasks
- **Smart Triggers:** Start flows based on specific events
- **Targeted Actions:** Execute multi-step tasks across different apps
- **Pre-built Templates:** Deploy common business workflows instantly

Dataverse – The Foundation

- **Secure Cloud Storage:** Store business data safely
- **Relational Database:** Link different tables together efficiently
- **Data Standardisation:** Use built-in schemas for common business needs
- **Role-Based Security:** Restrict data access based on user roles

Architecture

- **The Hub Concept:** Microsoft Dataverse sits at the centre, serving as the single source of truth for your data
- **User Interaction:** Power Apps acts as the user face, pushing and pulling information directly from Dataverse
- **Automated Action:** Power Automate listens for changes in Dataverse or Power Apps to launch background logic
- **Unified Loop:** A user action in Power Apps can immediately trigger a backend flow, updating Dataverse instantly



What is Event-Driven Programming?



Event-Driven Programming

- Traditional procedural code runs top to bottom; Power Apps instead reacts to events
- Every control has properties tied to events, such as OnSelect for a button

Code Example:

```
// Button.OnSelect property  
Set(varMessage, "Button was clicked!");  
Notify("Task submitted", NotificationType.Success)
```

Welcome to Power Fx!

How Power Fx Executes Code

- Power Fx is a low-code, strongly-typed, functional language built around declarative formulas and expressions
- Unlike rigid standalone programs, Power Fx expressions evaluate dynamically when values change or during linear automation flow steps

Example:

```
Trace("Step 1: Preparing ingredients.");
```

```
Trace("Step 2: Cooking the meal.");
```

```
Trace("Step 3: Dinner is served!");
```

Core Characteristics



**EXCEL-
INSPIRED**



**STRONGLY
TYPED**



DECLARATIVE



**CLOUD-
INTEGRATED**

Core Syntax Rules

Key Rules:

Rule	Description
Semicolons (;)	Used as an operator to chain multiple distinct actions together sequentially.
Parentheses	Semicolons ; terminate every statement, such as variable declarations and method calls
Quotation Marks	Round brackets () are used for calling methods like Console.WriteLine()
Curly Braces	Strings must be wrapped in matching double "" quotes; single " quotes are reserved for a single char

Consistency is key; a missing semicolon or brace will stop your code compiling

Right vs. Wrong (Syntax Comparison)

Correct Script:

```
If(true,  
    Notify("This is correct!"))
```

Flawed Script:

```
If(true  
    Notify("This is incorrect!"))
```

What are the 3 errors in the flawed version?

- Missing a comma separator after the condition,
- An unclosed double quotation mark on the text string,
- A missing closing parenthesis for the outer function wrapper

Corrected Script:

```
If(true,  
    Notify("This is now correct!"))
```

Power Automate: Event-Driven Workflows

- Like Power Apps, Power Automate is event-driven: a trigger starts the flow, then a sequence of actions runs automatically
- Conditions and expressions in a flow use Power Fx formulas, so the logic you learn in Power Apps carries across the platform

Code Example:

// Trigger: When an item is created (SharePoint / Dataverse)

// Condition action (Power Fx formula bar)

If(Amount > 1000, "Needs approval", "Auto-approved")

// Action: Send an email notification

Controls, Properties and Formulas

- Power Fx is the formula language used throughout Power Apps — and increasingly across Power Automate too — similar in spirit to Excel formulas
- A control's Text property can reference another control's value directly

Code Example:

// Label.Text property

"You entered: " & TextInput1.Text

Right vs. Wrong (Event Wiring)

Missing Event Handling:

```
// Button OnSelect left empty  
// nothing happens when clicked
```

Corrected — Wired to Logic:

```
// Button.OnSelect  
Set(varTaskCount, varTaskCount + 1);  
Notify("Task added")
```

Why does the app appear broken when OnSelect is left empty?

- Because leaving the OnSelect property empty means no code or logic is bound to the user's click action
- In event-driven environments like Power Apps, a button is just a visual element until you give it instructions

Guided Exploration Activity



ACTIVITY



Your Mission:

Open the sample application and complete a quick UI audit

Explore the Pre-Built App:

- Identify inputs, e.g. forms and text fields
- Identify outputs, e.g. labels and screens
- Trace how actions trigger behavior

Tasks:

- Describe "what happens when..." scenarios as prompted by the tutor



ACTIVITY





Next Activity will start soon

Task Entry App



ACTIVITY



Objective: Build a simple app that captures and displays a task

Tasks / Requirements:

- Add input fields for task name and description
- Add a submit button that stores the values
- Add a label that displays the submitted task

Key Point: Focus on understanding control properties and linking UI to logic

Extension Activity: Add validation so the button shows a warning if the task name is empty



ACTIVITY





Next Activity will start soon

Event Mapping Exercise



ACTIVITY



Map User Interface (UI) Actions to Logic:

- Button → submit
- Input → store value
- Output → display change

Tasks:

- Apply this mapping to the Task Entry App



ACTIVITY



Inline Debugging & State Inspection



Debugging Power Fx: Real-Time Variables

- **Formula Bar Result View:** Click any formula segment or collection block to reveal current runtime data structures instantly
- **The Alt Key Trick:** Hold down the **Alt key** to interact with controls directly in edit mode without playing the app
- **Inspecting Variables:** Navigate to the **Variables** panel via the side navigation menu to view global states, context variables, and collections
- **Visual Error Indicators:** Hover directly over red wavy syntax underlines to view contextual error messages and required data types

Advanced Diagnostics with Live Monitor

Advanced Troubleshooting: Power Apps Monitor

- **The Live Monitor:** Launch the native diagnostic dashboard from **Advanced Tools** to capture chronological user and backend event data streams
- **Network Request Visibility:** Inspect exact payloads, status success codes, and data payload properties returned by external connectors
- **The Trace() Function:** Write custom telemetry expressions directly into Power Fx actions to pass strategic variables into the live logging feed
- **Production Debugging:** Toggle on the "**Debug published app**" configuration setting to stream telemetry data from live production environments

Telemetry with the Trace() Function

Implementing Custom Telemetry with Trace()

- **Purpose:** Log custom data points directly to the Live Monitor stream when actions occur
- **Syntax:** Trace(Message [, TraceSeverity [, CustomRecord]])
- **Step 1:** Identify critical action points in your app (e.g., submitting a high-value purchase request)
- **Step 2:** Add the Trace() function inline to capture contextual details like usernames or current app states
- **Step 3:** Open the Monitor tool and trigger the action to watch your custom data stream in real time

```
// Button.OnSelect property with telemetry tracking
If(
    IsBlank(TextInput_AssetName.Text),
    // Log validation failure event
    Trace(
        "Validation Failed: Blank Asset Name",
        TraceSeverity.Warning,
        { userEmail: User().Email, Screen: App.ActiveScreen.Name }
    );
    Notify("Asset Name is required!", NotificationType.Error),
    // Log successful operation path
    SubmitForm(EditForm_AssetRequest);
    Trace(
        "Asset Submitted Successfully",
        TraceSeverity.Information,
        { AssetName: TextInput_AssetName.Text, SubmittedBy: User().Email }
    )
)
```

Debugging Activity



ACTIVITY



Open the file **Activity1** from the Git Repo

Objective: Evidence a structured approach to debugging

Tasks 1:

- Identify the cause of each issue

Tasks 2:

- Correct the control properties or formulas

Tasks 3:

- Re-test the app to confirm the fix



ACTIVITY





FIREBRAND

LYCEUM EDUCATION GROUP

Evaluation Discussion



Procedural vs. Event-Driven

Traditional (Procedural):

- Driven by the code. The system executes a predefined, top-to-bottom sequence of instructions until it reaches the end
- **Control Flow:** Linear
- **Lifecycle:** Finite
- **Blocking Behaviour:** High
- **State Handling:** Passed sequentially from function to function

Modern (Event-Driven):

- Driven by the user or system activity. The program sits in an idle state until an external trigger launches a specific block of logic
- **Control Flow:** Inverted
- **Lifecycle:** Infinite
- **Blocking Behaviour:** Low
- **State Handling:** Global or Contextual, reacting to sudden changes

The Power Platform Fit:

- Power Fx runs entirely on this modern approach, executing code only when specific states change or user events occur

Real-World Metaphors

Drive-Thru Restaurant:

1. You place your order
2. You drive to the window
3. You pay
4. You receive your food

If the kitchen slows down on step 3, the entire line of cars stops moving

No one can skip ahead or change their sequence

Tapas Restaurant:

1. You sit at a table
2. The kitchen is ready, but nothing happens until you place an order
3. While waiting for your first dish, you can order a drink
4. The dishes arrive at your table as soon as they are ready, in no specific chronological order

Which is Procedural and which is Event-Driven?

Drive-Thru Restaurant = *Procedural*

|

Tapas Restaurant = *Event-Driven*

Power Fx Application Context

- **The Formula Bar Reality:** In Power Apps, you do not write a single "main" script that runs your entire app
- **Isolated Logic:** Instead, you write tiny, independent chronological micro-scripts inside specific element properties (like Button.OnSelect or Screen.OnVisible)
- **The Listener:** The underlying Power Apps architecture runs the infinite event loop, listening for a user touch or data change to awaken those micro-scripts

Procedural vs. Power Fx

JavaScript:

// A single script manually controlling every sequential step

```
function handleClick() {  
    let inputField = document.getElementById("taskInput");  
  
    if (inputField.value === "") {  
        alert("Task cannot be blank!");  
        return; // Halts execution explicitly  
    }  
  
    let taskName = inputField.value;  
    saveToDatabase(taskName);  
  
    updateUIElement("statusLabel", "Saved!");  
    console.log("Telemetry: Task saved successfully.");  
}
```

Power Fx:

// Small, reactive micro-scripts bound directly to the user interaction event

```
If(  
    IsBlank(TextInput_TaskName.Text),  
    Notify("Task cannot be blank!", NotificationType.Error),  
  
    Patch(TasksCollection, Defaults(TasksCollection),  
        { Title: TextInput_TaskName.Text });  
  
    Notify("Saved!", NotificationType.Success);  
    Trace("Telemetry: Task saved successfully.")  
)
```

Stretch and Challenge

The Mission:

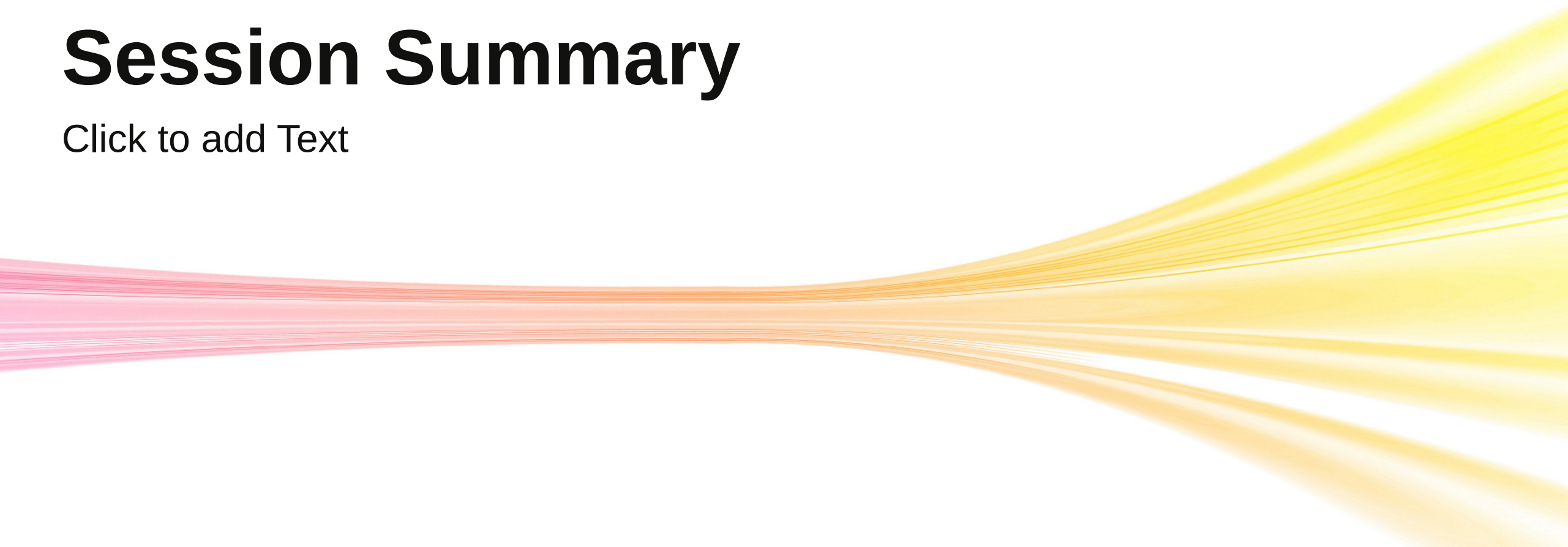
Improve your working logger application into production-grade code

- **Task:**

Add validation behaviour, e.g. an empty input warning

Session Summary

Click to add Text



Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, **S7**, S10,
S11, S12, S16

Behaviours

B2, B3

